



写出下列程序的运行结果

```
int main()
{  const char *c[]={ "John learn C++ language",
                      "Be well!", "You", "Not very" };

  const char **p[]={ c+3, c+2, c+1, c };
  const char ***pp=p;
  cout << (**++pp);
  cout << (*--*++pp+4);
  cout << (*pp[-2]+3);
  cout << (pp[-1][-1]+2);
  cout << endl;
  return 0;
}
```

- 注：直接在本文件上作答，**画出程序执行过程的内存变化**即可
- ★ 首先画出三句定义语句结束后内存中各变量的所占空间及初值
 - ★ 每个执行语句的每一步执行完成后的内存中各变量的所占空间及值
 - ★ 每步变化一个页面(例：**++pp，分三步计算，需要三页)
 - ★ **不允许**手写在纸上，再拍照贴图
 - ★ **允许**在各种软件工具上完成，再截图贴图
 - ★ 转换为pdf后提交



```
const char *c[]={  
    "John learn C++ language",  
    "Be well!", "You", "Not very"};
```

无名字串地址

c

2000	3000
2004	3100
2008	3200
2012	3300

c中指针的地址

3000	J
3001	o
3002	h
3003	n
3004	
3005	l
3006	e
3007	a
3008	r
3009	n
3010	
3011	C
3012	+
3013	+
3014	

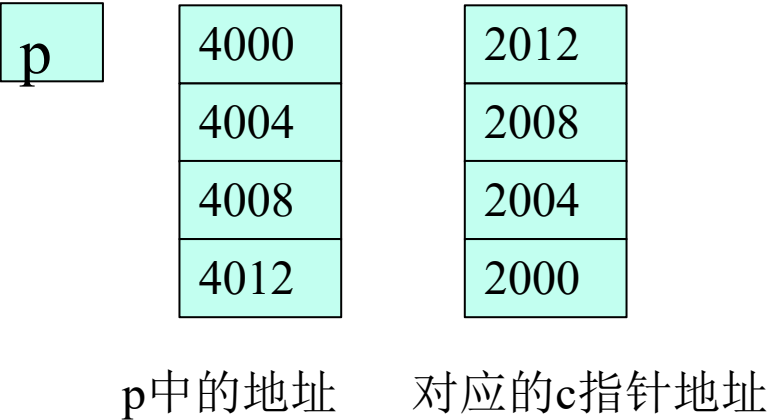
3100	B
3101	e
3102	
3103	w
3104	e
3105	l
3106	l
3107	!
3108	\0
3109	n

3200	Y
3201	o
3202	u
3203	\0

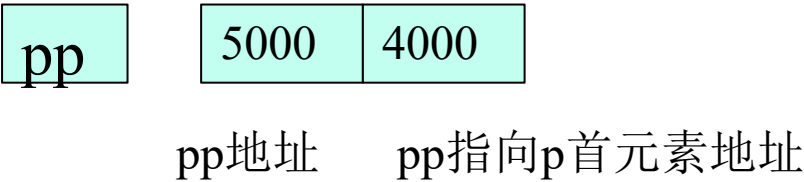
3300	N
3301	o
3302	t
3303	
3304	v
3305	e
3306	r
3307	y
3308	\0



```
const char **p[]={c+3, c+2, c+1, c};
```



```
const char ***pp=p;  
其实是const char** (*pp)=p
```





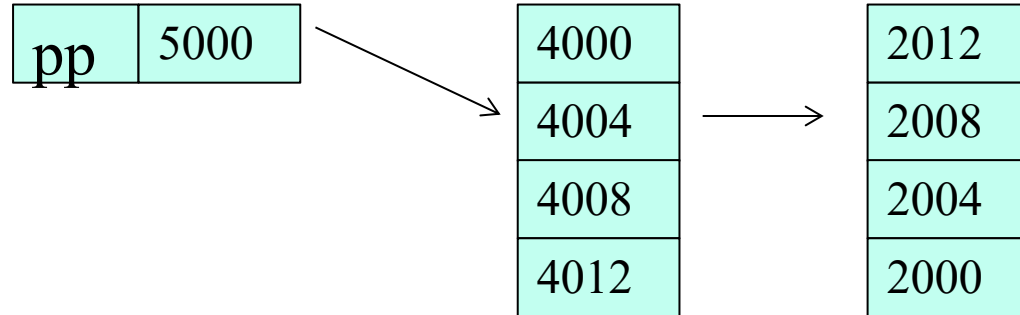
****++pp**

pp	5000	4004
----	------	------

先进行++操作，pp指向p[1]



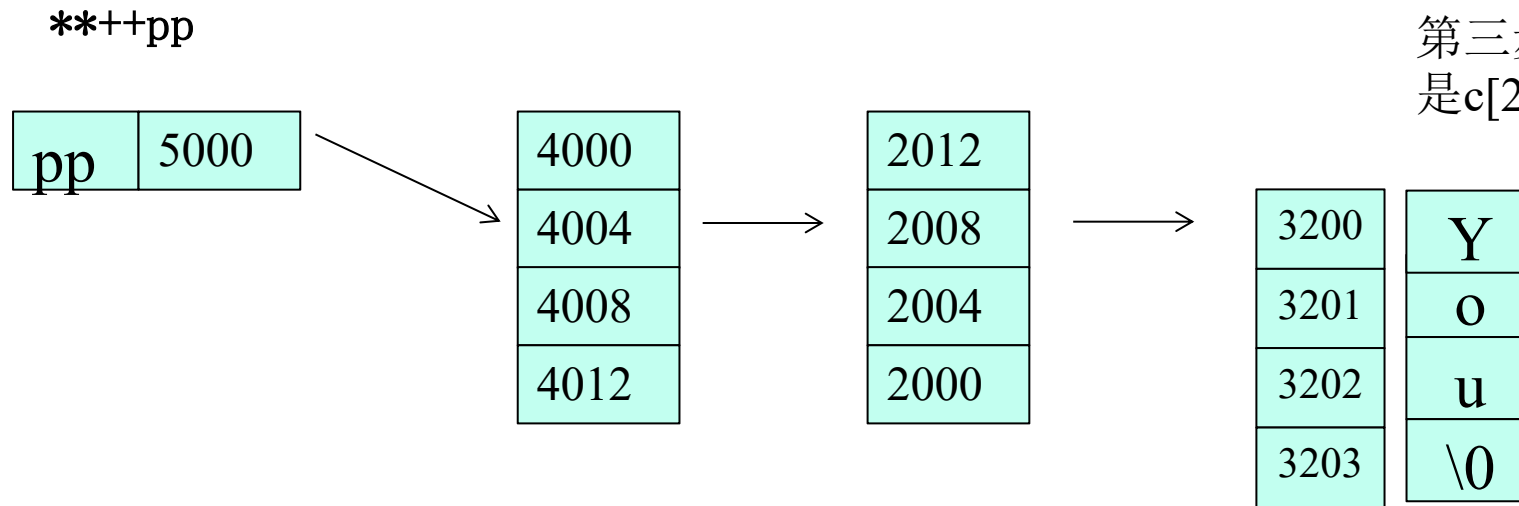
****++pp**



第二步，取p[1]的值，也就是c[2]的地址



第三步，取c[2]的值，也就是c[2]的地址



因此输出“You”

(*--*++pp+4)

pp	5000	4008
----	------	------

第一步, ++pp,指向p[2]

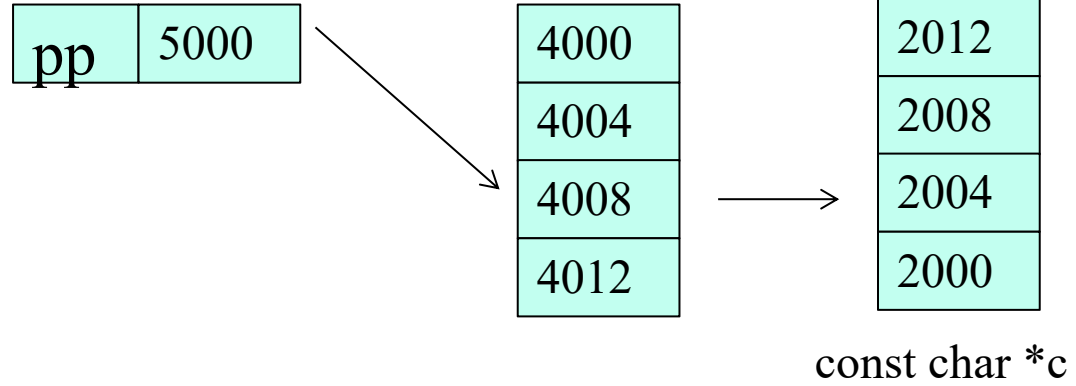




(*--*++pp+4)

const char **p[3]

第二步，取*，得到p[2]地址

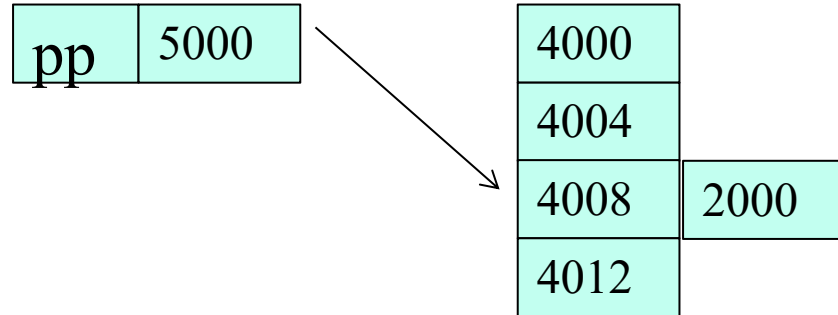




`(*--*++pp+4)`

`const char **p[3]`

第三步, --, p[2]指向2000

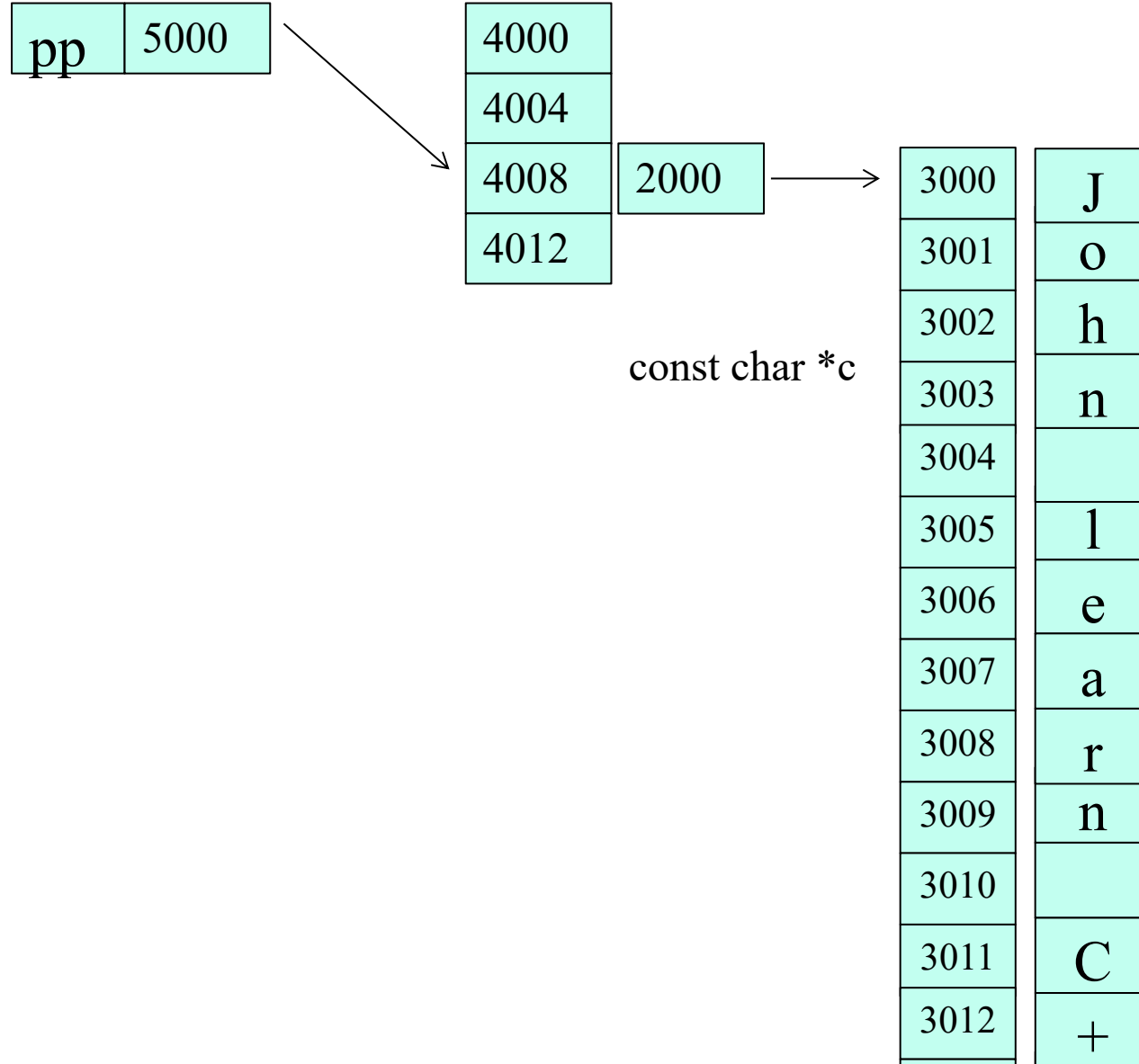




(*--*++pp+4)

const char **p[3]

第四步，取*c[0]，此时位置是c[0]

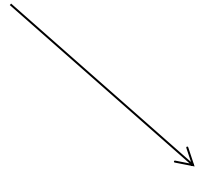




(*--*++pp+4)

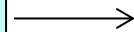
const char **p[3]

pp	5000
----	------



4000
4004
4008
4012

2000



const char *c

3000	J
3001	o
3002	h
3003	n
3004	
3005	l
3006	e
3007	a
3008	r
3009	n
3010	
3011	C
3012	+
3013	+
3014	

因此输出”learn c++ language”

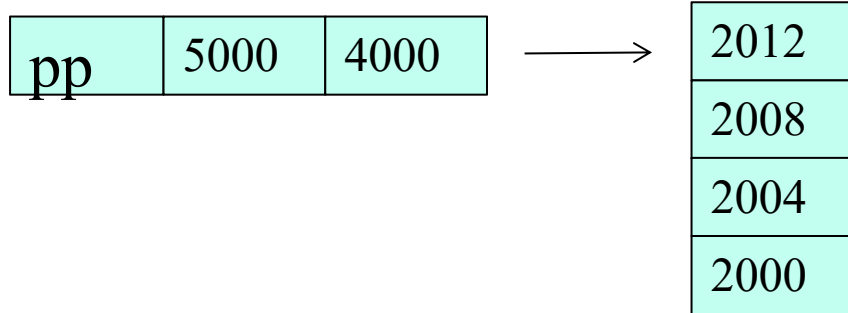
第五步，+4，指向2000的字符串的第四个字符

3015	l
3016	a
3017	n
3018	g
3019	u
3020	a
3021	g
3022	e
3023	\0



```
cout << (*pp[-2]+3);
```

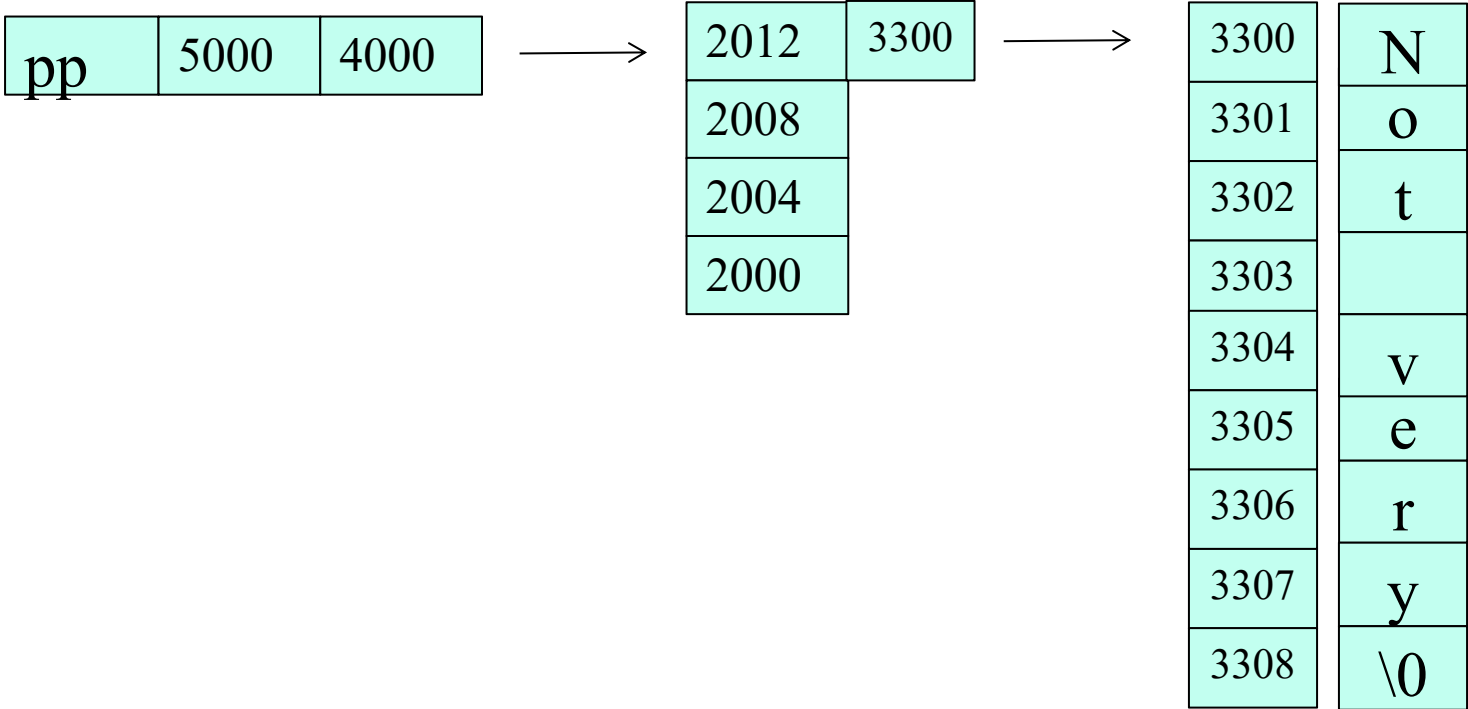
第一步，pp原先存有p[2]的地址，pp[-2]则即是
* (pp-2)





```
cout << (*pp[-2]+3);
```

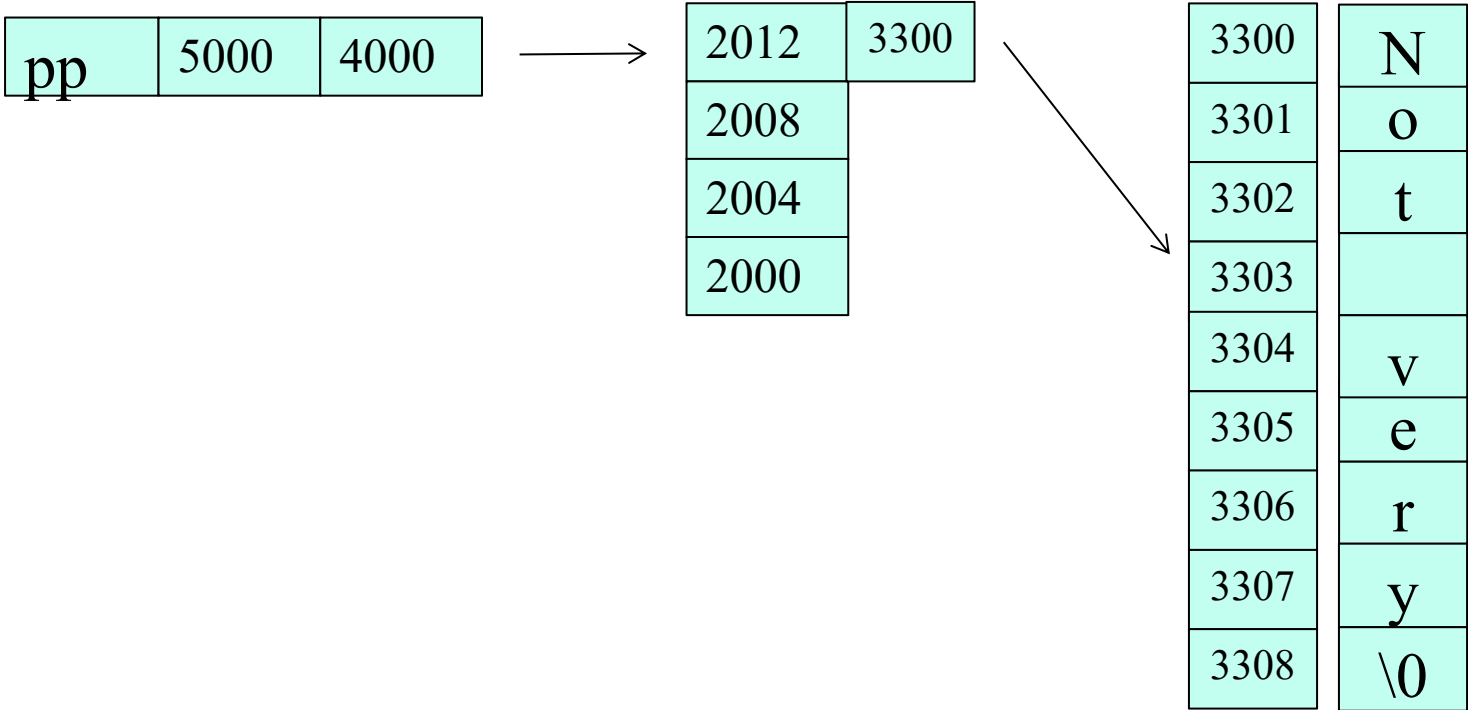
第二步，取*,也就是取2012指针里存的3300位置的字符串





```
cout << (*pp[-2]+3);
```

第三步， +3,此时的位置是3303



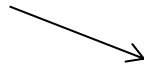
输出“ very”



```
cout << (pp[-1][-1]+2);
```

第一步, pp[-1],也就是*(pp-1),取的是4004的地址

pp	5000	4004
----	------	------



4000	2012
4004	2008
4008	2004
4012	2000

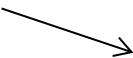
*p[] *p[]内存存储的值



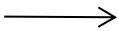
```
cout << (pp[-1][-1]+2);
```

第二步, pp[-1][-1],也就是* (*(pp-1)-1) ,取到c[1]
字符串

pp	5000	4004
----	------	------



4000	2012
4004	2008
4008	2004
4012	2000



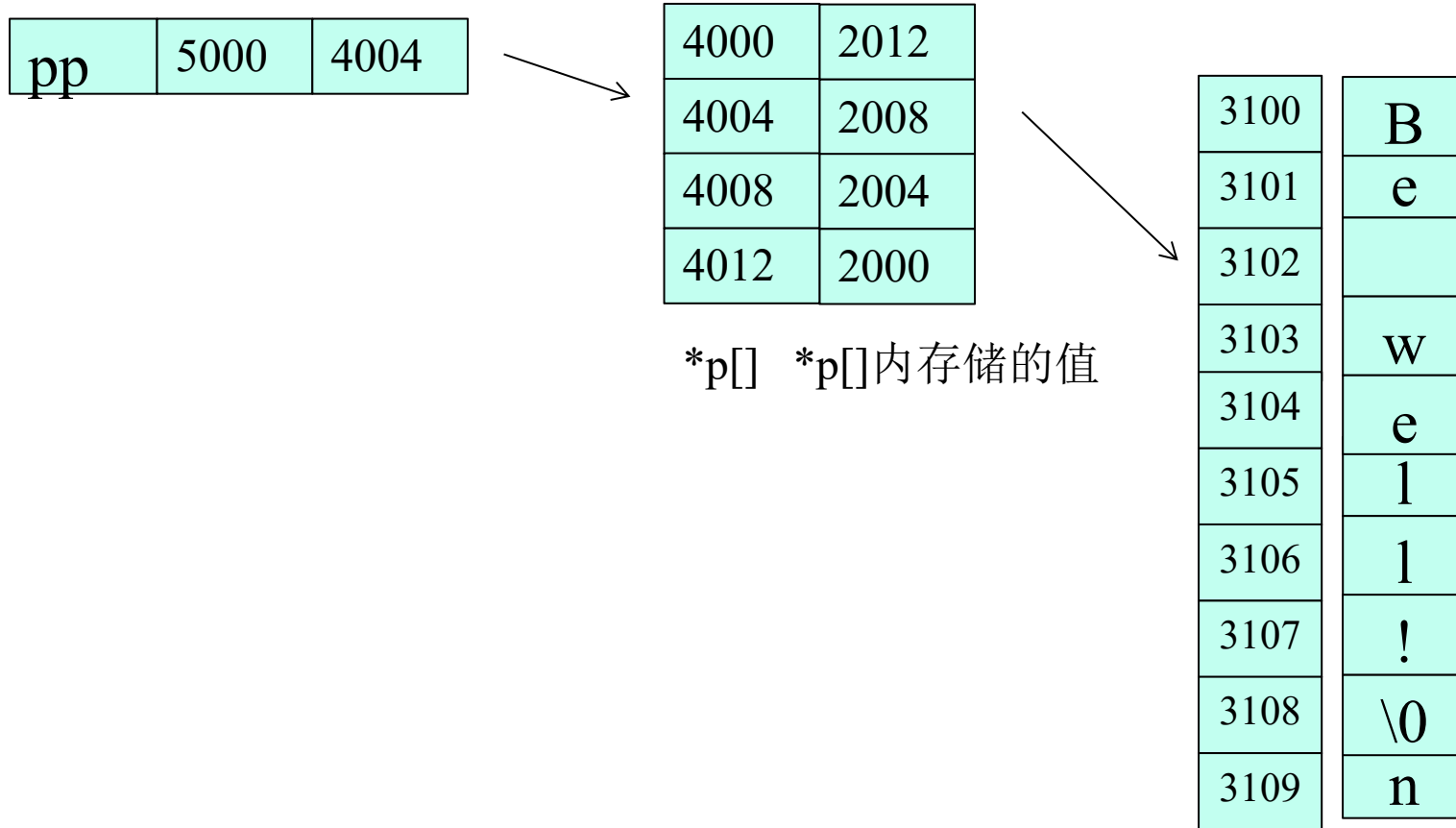
3100	B
3101	e
3102	
3103	w
3104	e
3105	l
3106	l
3107	!
3108	\0
3109	n

*p[] *p[]内存储的值



```
cout << (pp[-1][-1]+2);
```

第三步, +2,也就是从3102开始



*p[] *p[]内存存储的值

输出“ well!”

最后输出: “You learn c++ language very well!”